filter kernel, the output signal is $N+M-1$ samples long. For instance, the input signal, (a), is 300 samples (running from 0 to 299), the filter kernel, (b), is 101 samples (running from 0 to 100), and the output signal, (i), is 400 samples (running from 0 to 399).

In other words, when an N sample signal is filtered, it will be *expanded* by $M-1$ points *to the right*. (This is assuming that the filter kernel runs from index 0 to M. If negative indexes are used in the filter kernel, the expansion will also be to the *left*). In (a), zeros have been added to the signal between sample 300 and 399 to illustrate where this expansion will occur. Don't be confused by the small values at the ends of the output signal, (i). This is simply a result of the windowed-sinc filter kernel having small values near its ends. All 400 samples in (i) are nonzero, even though some of them are too small to be seen in the graph.

Figures (c), (d) and (e) show the decomposition used in the overlap-add method. The signal is broken into segments, with each segment having 100 samples from the original signal. In addition, 100 zeros are added to the right of each segment. In the next step, each segment is individually filtered by convolving it with the filter kernel. This produces the output segments shown in (f), (g), and (h). Since each input segment is 100 samples long, and the filter kernel is 101 samples long, each output segment will be 200 samples long. The important point to understand is that the 100 zeros were added to each input segment to allow for the expansion during the convolution.

Notice that the expansion results in the output segments *overlapping* each other. These overlapping output segments are added to give the output signal, (i). For instance, samples 200 to 299 in (i) are found by adding the corresponding samples in (g) and (h). The overlap-add method produces exactly the same output signal as direct convolution. The disadvantage is a much greater program complexity to keep track of the overlapping samples.

FFT Convolution

FFT convolution uses the principle that *multiplication* in the frequency domain corresponds to *convolution* in the time domain. The input signal is transformed into the frequency domain using the DFT, multiplied by the frequency response of the filter, and then transformed back into the time domain using the Inverse DFT. This basic technique was known since the days of Fourier; however, no one really cared. This is because the time required to calculate the DFT was *longer* than the time to directly calculate the convolution. This changed in 1965 with the development of the Fast Fourier Transform (FFT). By using the FFT algorithm to calculate the DFT, convolution via the frequency domain can be *faster* than directly convolving the time domain signals. The final result is the same; only the number of calculations has been changed by a more efficient algorithm. For this reason, FFT convolution is also called **high-speed convolution**.